

ACMX — Developer Documentation

The website of the **FEU Tech ACM Student Chapter**. This document explains what every part of the codebase does, where the seams are, and — for each feature — **which files you edit when you redesign it**.

Read the "Orientation" and "Design system" sections first. After that the per-feature sections stand alone; jump to the one you're working on.

CONTENTS

1 Orientation

2 Running it locally

3 Architecture

4 Authentication, sessions and roles

5 The design system

6 Data model

7 Feature map

8 Conventions

9 Common tasks

10 Known rough edges

1. Orientation

Framework	Next.js 16 (App Router, React 19, Server Components by default)
Language	TypeScript, strict-ish; path alias <code>@/*</code> → <code>src/*</code>
Styling	Tailwind CSS v4 plus a hand-rolled token system (<code>src/styles/design-system.ts</code>)
Database	PostgreSQL via Prisma 6 (<code>prisma/schema.prisma</code>)
File storage	Supabase Storage (4 public buckets)
Auth	Custom: bcrypt password + opaque DB session, <code>session</code> httpOnly cookie
Animation	GSAP (a shared "blink in" reveal)
Deployment	Vercel (<code>output: "standalone"</code>)

Two visual worlds live in this repo, and knowing which one you're in matters:

- **The editorial-brutalist system** — concrete surfaces, Monument Extended display type, hairline rules, hard 90° corners, one orchid accent. This is the current design language. It is driven by `src/styles/design-system.ts` and the primitives in `src/components/ds/`. Everything new should be built here.
- **The legacy Tailwind theme** — the CSS variables in `src/app/globals.css` (`--surface` , `--dm-border` , `--accent` , ...). Older components still consume these through Tailwind utility classes like `bg-surface text-dm-text-primary`. Don't extend it; port screens to the DS as you touch them.

2. Running it locally

```
npm install          # postinstall runs `prisma generate`
cp .env.example .env # then fill in the four values
npx prisma migrate dev # apply migrations to your database
npm run dev          # http://localhost:3000
```

Environment variables (all documented inline in `.env.example`):

VARIABLE	READ BY	SECRET?
DATABASE_URL	prisma/schema.prisma	yes
NEXT_PUBLIC_SUPABASE_URL	src/lib/supabase.ts, upload route	public
NEXT_PUBLIC_SUPABASE_ANON_KEY	src/lib/supabase.ts	public
NEXT_SUPABASE_SERVICE_ROLE_KEY	src/app/api/upload/route.ts server only	yes — bypasses RLS

Anything prefixed `NEXT_PUBLIC_` is inlined into the browser bundle. Never put a secret behind that prefix.

Supabase Storage buckets — `merch`, `events`, `eventCard`, `videos`. All public-read. The allowlist lives in `src/app/api/upload/route.ts`; adding a bucket means creating it in Supabase *and* adding the name there.

Seeds (`prisma/`, run with `node`):

SCRIPT	SEEDS
seed-committees.mjs	The chapter's nine committees, upserted by slug. Safe to re-run. Rosters intentionally empty.
seed-events.mjs	Sample events
seed-merch.mjs	Sample merch catalogue
seed-members-2526.mjs, seed-members.ts	Member accounts
scripts/make-admin.js	One-off role promotion

Scripts: `npm run dev`, `npm run build` (uses `--webpack`, not Turbopack), `npm start`, `npm run lint`.

3. Architecture

Request path

```
Browser
|
|─ src/middleware.ts ..... signed-in check on /dashboard /profile /settings /admin,
|                           guest-only bounce on /hero, stamps `x-pathname`
|
|─ src/app/**/page.tsx ..... Server Component. Reads the session, fetches via Prisma
|                           or passes props down. Usually thin.
|   |
|   └─ src/components/** Client Component. Owns interaction + `fetch()` to the API.
|
|─ src/app/api/**/route.ts ... Route Handler. Re-authenticates, re-validates,
|                           talks to Prisma, returns JSON.
```

The layering rule

Pages are thin, components are fat, routes are the security boundary.

- `page.tsx` files typically do three things: resolve the current user, do a redirect if needed, and render one client component. Look at `src/app/dashboard/page.tsx` or `src/app/merchandise/page.tsx` — that's the shape.
- All interactivity lives in `src/components/<feature>/`.
- Every API route calls `getCurrentUser(req)` itself and re-checks the role. Middleware only proves *someone is signed in* — it never proves *they may do this*. Client-side validation is a courtesy; the route handler is the gate.

Directory map

```
src/
  app/
    <route>/page.tsx      Public + member pages
    admin/                Admin console (role-gated by admin/layout.tsx)
    api/                  Route handlers – the JSON API
    layout.tsx            Root layout: fonts, ThemeProvider, NavBar, theme anti-flash script
    globals.css           Legacy CSS-variable theme + Tailwind import
  components/
    ds/                   Design-system primitives – the shared vocabulary
    admin/                Admin console screens
    events/ merch/ committee/ dashboard/ profile/ about/ officers/ login/ UI/
  lib/                    Server + shared helpers (Prisma, auth, serializers, validation)
  services/               Older service layer (identity, attendance)
  styles/design-system.ts The token file. The single source of visual truth.
  types/                  Shared enums, labels and DTO shapes
  middleware.ts
  prisma/                 schema.prisma, migrations/, seed scripts
  public/                 assets, bg, fonts, events, eventCard, references
```

`lib/` vs `services/`: `services/` predates `lib/` and holds the identity and attendance logic. New shared logic goes in `lib/`.

`proxy.ts` and `server.js` at the repo root are **not wired into the app**. `proxy.ts` is a superseded copy of the middleware; `server.js` is a custom standalone server. Ignore both unless you're changing hosting.

4. Authentication, sessions and roles

The flow

1. `POST /api/login` → `services/identity/identityService.ts` → `login()` verifies the bcrypt hash, `createSession()` writes a `Session` row.
2. The route sets an `httpOnly` `session` cookie holding the session id (7 days, `sameSite: lax`, `secure` in production).
3. `getCurrentUser()` in `src/lib/auth.ts` reads the cookie, looks the session up, checks `expiresAt`, and returns the full Prisma `User` — or `null`.

`getCurrentUser` is the single source of truth. It works in two modes: pass the `NextRequest` inside an API route; call it with no argument inside a Server Component (it falls back to `next/headers`).

Never hand a raw `User` to a client component — run it through `toSafeUser()` (`src/lib/userMapper.ts`), which strips the password hash and flattens the name. The client-side shape is `safeUser` in `src/types/auth.ts`.

Roles

The `UserRole` enum in `prisma/schema.prisma` is the source; `src/types/auth.ts` turns it into tiers, labels and predicates. **Edit both when adding a role.**

HELPER	GRANTS
<code>isAdmin(role)</code>	<code>ADMIN</code> , <code>PRESIDENT</code> , <code>VP_INTERNAL</code> , <code>VP_EXTERNAL</code> — the "top table". Full run of <code>/admin</code> .
<code>isEventAdmin(role)</code>	Top table + executives, secretariat, media officer. Create/edit events, upload files.
<code>isSecretariatOrAbove(role)</code>)	On-site registration.
<code>isOfficer(role)</code>	Any officer — used for the officer pricing tier.
<code>roleLabel(role)</code>	Human-readable name.

Every gate tests membership of a list, never `role === "ADMIN"`. Keep it that way.

Committee-scoped access

`src/lib/committee-access.ts` adds a second, orthogonal axis so committee leads can run their own committee without being admins:

- **EDIT** — the top table on every committee, plus a committee's own head/co-head (or, on the DEV track, project lead/lead dev) on *that* committee.
- **VIEW** — anyone else holding a seat, read-only.
- **NONE** — everyone else.

`committeeScope(user)` returns `{ admin, seats, access(id), hasAny }` and is the reusable answer for a request. A lead's rights stop at **content and roster** — creating, deleting, reordering and publishing stay with the top table.

`src/app/admin/layout.tsx` enforces this: admins pass through; seat-holders are redirected to `/admin/committees` and no further (it reads the `x-pathname` header the middleware stamps, because a layout can't see the request path).

5. The design system

src/styles/design-system.ts — the token file

This is where you change how the whole site looks. It exports:

EXPORT	WHAT IT CONTROLS
accent	The single orchid accent, tuned per theme (light #9B2FBE , dark #CF78EC)
palette	surface , panel , text , muted , faint , rule , ruleStrong per theme
status	danger / positive (+ washes). State only, never decoration.
resolvePalette(theme)	Merges the three above into the flat Palette object components use
texture	The concrete grain image + per-theme blend mode
font	display = Monument Extended, body = Helvetica Now MT Text
type	eyebrow , display , title , heading , subheading , body , bodySmall , label , mono — every size is a clamp() , so type scales off the viewport with no breakpoints
layout	gutter (7vw), navHeight , topPad , bottomPad , gap , gapTight , measure
motion	fast / base durations and the shared easing curve

Changing a colour, a type scale or the page gutter is a one-line edit here and it propagates everywhere. Resist hardcoding hex values or px sizes in components.

Fonts live in public/fonts/ and are declared in globals.css ; next/font loads Geist as the fallback family in layout.tsx .

src/components/ds/ — the primitives

Import from the barrel: import { Surface, Panel, Button } from "@components/ds" .

PRIMITIVE	ROLE	NOTABLE PROPS
Surface	The page substrate — textured background + optional edge-bleed diamond motion. Every page sits in one.	corners: "both" "top-left" "bottom-right" "none" , fullHeight
Column	Centered content column with the standard gutters and top/bottom padding	
PageHeader	The masthead every interior page opens with: eyebrow → rule → title → rule → intro	eyebrow (array spreads edge-to-edge), title , intro , aside
Panel	Hairline-bordered block. Square corners, no shadow — depth comes from the border, never elevation	interactive (border warms to accent on hover), padded

PRIMITIVE	ROLE	NOTABLE PROPS
Badge , DataRow	Small status chip; label/value row	
Button	Square-cornered, wide-tracked	variant: "solid" "outline" "ghost", block
Field	Text input, underline (on the substrate) or boxed (in modals/panels). error takes over the border, prints the message, and wires role="alert" + aria-describedby	label , id , variant , error , hint , trailing
Segmented	Segmented control / tab picker	
Modal	The dialog. Focus trap, focus return, Esc + scrim dismiss, dirty guard against losing typed input, scroll lock, rendered in a portal	open , onClose , title , message , footer , dirty , width
Text exports	Rule , Eyebrow , Display , Title , Heading , Subheading , Body , Label	
useDS()	The hook every DS component uses. Returns { c , isDark } where c is the resolved palette for the active theme	

The pattern throughout is inline `style` objects fed from `useDS()` and the token file — not Tailwind classes — because the palette is theme-resolved at runtime in JS. Tailwind is still used for layout utilities (`flex` , spacing, responsive prefixes).

Theme

`src/components/ThemeProvider.tsx` holds `light` | `dark` in state, persists to `localStorage` , and toggles `.dark` on `<html>` . An inline script in `layout.tsx` applies the stored theme before paint to prevent a flash. Consume it with `useTheme()` for the toggle, or `useDS()` for colours.

Motion

`src/lib/blink.ts` exports `runBlinkIn(targets, opts)` — the site's shared reveal: a quick staggered opacity flicker, no movement, matching the preloader and login glitch. It respects `prefers-reduced-motion` . `Surface` and `Modal` call it automatically; `BlinkIn` wraps arbitrary children.

Other shared UI: `WithPreLoader` , `Animation` , `ContinuousAnimation` , `DiamondScaleBackground` , `NavBar` , `ProfileMenu` — all in `src/components/UI/` .

6. Data model

`prisma/schema.prisma` is heavily commented — read it alongside this summary.

Identity — `User` (`studentId` is the natural key; `role`, `points`), `Session`, `Schedule`, `Transaction`.

Events — `Event` self-relates via `parentId` / `subEvents` so a multi-day event is a parent with one child per day. Carries three price tiers, a card image, a gallery, an optional `statusOverride`, and extended detail fields (`overview`, `mainObjective`, `specificObjectives`, ...). `Registration` (uniquely keyed per event by user, email *and* student number) → `Attendance` (one per registration, `timeIn` / `timeOut`).

Merchandise — `MerchItem` → `MerchVariant` (a size/colour, with `stock` and an independent `soldOut` flag). `MerchCartLine` persists the basket server-side so it survives devices and can be re-validated at checkout. `MerchOrder` + `MerchOrderLine` are **reservations, not payments** — checkout decrements stock, holds it until `expiresAt`, and issues a `reference` the member quotes at the ACM room. Order lines snapshot name/label/price so editing the catalogue never rewrites what someone reserved. `MerchRestockRequest` is "notify me".

Committees — `Committee` plus four child tables: `CommitteeResponsibility`, `CommitteeProject`, `CommitteeFact` (free-form key/value, so a committee with no co-head simply has no co-head row), and `CommitteeMember` (the roster; `userId` links a real account, and *that link is what grants console access* — `name` is stored alongside so the roster survives account deletion). `track` (`STANDARD` | `DEV`) decides which role set the roster offers.

Content — `FeaturedVideo` (dashboard hero slideshow), `Lesson`.

Migrations

Four, in `prisma/migrations/`: `baseline`, `merchandise`, `committees`, `committee_roles_and_exec`. Standard Prisma workflow — `npm run prisma migrate dev --name <thing>` after editing the schema, and commit the generated SQL.

7. Feature map

Each subsection lists **what it is**, **the files**, and **what to edit when you redesign it**.

7.1 Landing, navigation and shell

FILE	ROLE
<code>src/app/page.tsx</code>	Pure redirect — signed in → <code>/dashboard</code> , else → <code>/hero</code>
<code>src/app/hero/page.tsx</code>	The public landing page. Guest-only (middleware bounces signed-in users). The visual reference the whole DS was derived from.
<code>src/app/layout.tsx</code>	Root: fonts, metadata, theme anti-flash script, <code>ThemeProvider</code> , <code>NavBar</code>
<code>src/components/UI/NavBar.tsx</code>	Fixed top nav. Links list, theme toggle, cart badge, <code>ProfileMenu</code> . Suppressed on <code>/admin</code> .
<code>src/components/UI/ProfileMenu.tsx</code>	Signed-in dropdown
<code>src/components/login/</code>	<code>LoginButton</code> , <code>LoginModal</code> , <code>login.tsx</code>

Redesigning the nav → `NavBar.tsx` . The links array (`BASE_LINKS`) is at the top of the file; "Home" is prepended per-user so the link skips the `/` redirect.

7.2 Dashboard (signed-in home)

FILE	ROLE
<code>src/app/dashboard/page.tsx</code>	Gate + <code>WithPreloader</code> wrapper
<code>src/components/dashboard/DashboardHome.tsx</code>	The whole screen — hero slideshow + cards
<code>src/components/dashboard/VideoCarousel.tsx</code>	Hero slideshow, driven by <code>FeaturedVideo</code>
<code>src/components/dashboard/CalendarCard.tsx</code> , <code>SchoolCalendarModal.tsx</code>	Calendar
<code>src/components/dashboard/ManageVideosModal.tsx</code>	In-place video management for officers
API	<code>GET/POST /api/videos</code> , <code>/api/videos/[id]</code>

Redesigning the dashboard → `DashboardHome.tsx` for layout, `VideoCarousel` for the hero.

7.3 Events

The largest feature. Events are listed by semester, opened individually, and have an admin panel for registration and attendance.

FILE	ROLE
<code>src/app/events/page.tsx</code>	List page shell → <code>components/events/Events.tsx</code>
<code>src/app/events/[eventId]/page.tsx</code>	One event
<code>src/app/events/[eventId]/admin/page.tsx</code>	Per-event officer panel
<code>components/events/Events.tsx</code>	Semester switcher + list orchestration
<code>components/events/EventsList.tsx</code> , <code>EventCards.tsx</code>	The list and its cards
<code>components/events/SelectedEvent.tsx</code>	The event detail view
<code>components/events/PastEventExperience.tsx</code>	The finished-event treatment (gallery, recap)
<code>components/events/EventGallery.tsx</code>	Image gallery
<code>components/events/EventCreationModal.tsx</code>	Create flow
<code>components/events/AdminEventPanel.tsx</code> , <code>AdminDaySelector.tsx</code>	Officer controls; day picker for multi-day events
<code>components/events/AttendanceLookup.tsx</code> , <code>AttendButton.tsx</code>	Attendance
<code>components/registration/RegistrationModal.tsx</code>	Registration form
<code>src/lib/events.ts</code>	Server-side queries, semester normalisation ("1st" / FIRST), aggregated registration counts for parent events
<code>src/types/events.ts</code>	<code>EventWithCount</code> and friends
<code>src/services/attendance/attendanceService.ts</code>	Time-in/time-out rules (must be registered; time-in required before time-out; <1h earns no points)

API routes under `src/app/api/events/`: `create`, `ongoing`, `semester/[semester]`, and per-event `route.ts`, `edit`, `status`, `registrations`, `check-registration`, `attendance`, `attendance/manual`, `attendance/stream` (SSE for live attendance), `attendance-lookup`. Also `/api/registrations`, `/api/registrations/complete`, `/api/registration-prefill`.

Redesigning → the list is `EventsList` + `EventCards`; the detail page is `SelectedEvent` (upcoming/ongoing) and `PastEventExperience` (finished). Pricing tiers come from the `Event` record and the viewer's role via `isOfficer`.

7.4 Merchandise

Reservation-based store: no online payment, no card details. Members reserve; officers fulfil in the ACM room.

FILE	ROLE
<code>src/app/merchandise/page.tsx</code>	Storefront shell
<code>src/app/merchandise/[slug]/page.tsx</code>	Item page
<code>src/app/merchandise/cart/page.tsx</code>	Cart
<code>components/merch/Storefront.tsx</code>	Grid, filters, empty states
<code>components/merch/ProductCard.tsx</code>	Card (first image is the cover; <code>isNewDrop</code> drives the accent)
<code>components/merch/ItemDetail.tsx</code>	Gallery, variant picker, facts list, notify-me
<code>components/merch/CartPage.tsx</code>	Cart + checkout
<code>components/merch/cartClient.ts</code>	<code>useCart()</code> — the cart client. The server owns the truth, so every mutation returns the re-validated basket; a <code>window</code> event (<code>acmx:cart</code>) keeps the nav badge in step without a root provider.
<code>src/lib/merch.ts</code>	Serializers (<code>serializeVariant</code> , <code>serializeOrder</code>), <code>loadCart</code> , <code>holdExpiry</code> , <code>orderReference</code> , sold-out derivation
<code>src/types/merch.ts</code>	DTOs, <code>MERCH_CATEGORIES</code> , <code>MERCH_STATUSES</code> , <code>HOLD_DAYS</code> , <code>slugify</code>

API: `/api/merch/items` , `items/[slug]` , `cart` , `checkout` , `orders` , `orders/[id]` , `notify` . Admin: `/api/admin/merch/items(/[id])` , `orders(/[id])` .

Notes worth knowing before you change anything here: an item is sold out when the admin says so **or** when every variant runs dry, so the store is self-maintaining. Checkout runs in a Prisma transaction and reports a lost stock race per line (`OutOfStock`), not as a blanket 500.

Redesigning → cards in `ProductCard` , the item page in `ItemDetail` , the grid/filters in `Storefront` . Cart *behaviour* is `cartClient.ts` ; cart *layout* is `CartPage.tsx` .

7.5 Committees

Every committee — the nine that exist and any admin-added one — renders through **the same page**, `/committee/[slug]` . The record supplies content; the design fixes section order and hierarchy. That is deliberate: it's what stops nine committees from becoming nine different websites.

FILE	ROLE
<code>src/app/committee/page.tsx</code>	Index — queries published committees directly via Prisma
<code>src/app/committee/[slug]/page.tsx</code>	The one reusable plate

FILE	ROLE
<code>components/committee/CommitteeIndex.ts</code> <code>x</code> , <code>CommitteeCard.tsx</code>	Index
<code>components/committee/CommitteePlate.ts</code> <code>x</code>	The whole detail page
<code>components/committee/Diamond.tsx</code>	The emblem mark
<code>src/lib/committee.ts</code>	<code>committeeInclude</code> (the canonical include), <code>serializeSummary</code> , <code>serialize</code> , recruiting reconciliation
<code>src/lib/committee-access.ts</code>	The EDIT/VIEW/NONE model (see §4)
<code>src/types/committee.ts</code>	Enums, labels, notes, <code>COMMITTEE_EMBLEMS</code> , <code>TRACK_POSITIONS</code> , <code>LEAD_POSITIONS</code> , <code>POSITION_RANK</code> , DTOs

Emblems are a **fixed set** (`COMMITTEE_EMBLEMS`), not uploads — the diamond mark has to read at 230px, so admins pick rather than upload. Add a new one in `types/committee.ts` and render it in `Diamond.tsx` .

Redesigning a committee page → `CommitteePlate.tsx` only. All nine change together, which is the point. Copy and labels live in `types/committee.ts` (`RECRUITING_LABELS` , `MEMBER_ROLE_LABELS` , `STATUS_NOTES` , ...) rather than inline in the component — edit them there so the console and the public page stay in agreement.

7.6 Profile / account

FILE	ROLE
<code>src/app/profile/page.tsx</code>	Gate; wraps in <code>Suspense</code> because the tab is read from <code>?tab=</code>
<code>src/app/settings/page.tsx</code>	Redirect to <code>/profile?tab=account</code> — kept so old links don't 404
<code>components/profile/AccountPage.tsx</code>	Tab shell
<code>components/profile/OverviewTab.tsx</code> , <code>AccountTab.tsx</code> , <code>SecurityTab.tsx</code>	The three tabs
<code>components/profile/ChangePasswordModal.tsx</code>	Password change
<code>components/profile/shared.tsx</code>	Local shared bits
API	<code>/api/me</code> , <code>/api/profile</code> , <code>/api/change-password</code> , <code>/api/sessions</code> , <code>/api/logout</code>

7.7 About and Officers

FILE	ROLE
<code>src/app/about/page.tsx</code> + <code>components/about/*</code>	<code>AboutHero</code> , <code>Mission</code> , <code>Vision</code> , <code>StudentChapter</code> , <code>Section</code> , <code>ContactFooter</code>

FILE	ROLE
<code>src/app/officers/page.tsx</code> + <code>components/officers/OfficersRoster.tsx</code>	The character-select style roster
<code>components/officers/officers-data.ts</code>	The roster content — hardcoded, not database-backed. Edit here to update officers.
<code>components/placeholder/ComingSoon.tsx</code>	Placeholder for unbuilt pages

7.8 Admin console

Lives at `/admin`, has its own chrome (the public `NavBar` is suppressed), and is gated by `src/app/admin/layout.tsx` (see §4).

SCREEN	PAGE	COMPONENT	API
Overview	<code>admin/page.tsx</code>	<code>admin/Overview.tsx</code>	<code>/api/admin/stats</code>
Events	<code>admin/events/page.tsx</code> , <code>events/[eventId]/page.tsx</code>	<code>EventsManager.tsx</code> , <code>EventEditor.tsx</code>	<code>/api/admin/events</code> , <code>/api/events/*</code>
Committees	<code>admin/committees/page.tsx</code> , <code>committees/[id]/page.tsx</code>	<code>CommitteesManager.tsx</code> , <code>CommitteeEditor.tsx</code> , <code>committee-</code> <code>ui.tsx</code>	<code>/api/admin/committees(/[id]</code> <code>)</code> , <code>/api/admin/members/search</code>
Merchandise	<code>admin/merchandise/page.tsx</code>	<code>MerchandiseManager.tsx</code>	<code>/api/admin/merch/*</code>
Media Library	<code>admin/media/page.tsx</code>	<code>MediaLibrary.tsx</code>	<code>/api/admin/media</code> , <code>/api/upload</code>
People & Roles	<code>admin/people/page.tsx</code>	<code>PeopleRoles.tsx</code>	<code>/api/admin/users</code> , <code>/api/admin/users/[studentNumber](/role)</code>
Videos	<code>admin/videos/page.tsx</code>	<code>VideosManager.tsx</code>	<code>/api/videos(/[id])</code>

`src/components/admin/AdminShell.tsx` is the console frame: persistent left sidebar, slim top bar with search, concrete surface. **The sidebar's `NAV` array at the top of that file is where you add a console section.** Icons come from `src/components/admin/icons.tsx` (`Icon` + the `IconName` union) — add the SVG there first.

The People & Roles picker is driven by `USER_ROLES` in `src/types/auth.ts`; add a role to the Prisma enum and to that array and it appears automatically.

7.9 Uploads and media

`POST /api/upload` is the only write path to storage. It authenticates, requires `EVENT_ADMIN_ROLES`, caps files at 50 MB, allowlists the bucket (`events` | `eventCard` | `videos` | `merch`), writes with the **service-role** key (server-side, bypassing RLS), and returns public URLs. Filenames are randomised on write.

Client-side reads use the anon-key client in `src/lib/supabase.ts`.

8. Conventions

Server vs client. Default to a Server Component. Add `"use client"` only when you need state, effects, or event handlers. Anything importing `useDS()` is client-side by definition.

Dynamic rendering. Pages and routes reading the session export `export const dynamic = "force-dynamic"`. Forget it and Next will try to statically render a page that depends on a cookie.

Validation. `src/lib/validation.ts` is imported by **both** the form and the route handler — that's what stops the two from drifting. Rules return `null` on success or a human-readable message written for the person who has to fix it, never "invalid input". `Validated<T>` is the result type.

Serialization. Prisma models never cross the wire raw. Each feature has a serializer (`lib/merch.ts` , `lib/committee.ts` , `lib/userMapper.ts`) producing the DTOs declared in `src/types/`. When you add a field, update the schema, the serializer, and the DTO.

Labels and copy live in `src/types/*.ts` as `*_LABELS` / `*_NOTES` records next to their enum — so the admin console and the public page can never disagree about what a status means.

Errors from API routes are `{ error: string }` with a real status code, and the message is written to be shown to a user.

Timezone. `getPhilippineTime()` in `src/lib/timezone.ts` — use it for anything attendance- or schedule-related.

Accessibility isn't optional in the primitives. `Modal` traps and returns focus; `Field` announces errors via `role="alert"` and `aria-describedby`; `runBlinkIn` respects `prefers-reduced-motion`. If you build a bespoke dialog or input instead of using the primitive, you inherit the obligation to redo all of that — which is why you should use the primitive.

9. Common tasks

Restyle the entire site → `src/styles/design-system.ts`. Colour, type scale, gutter, motion — all of it.

Redesign one page → find it in §7, edit the named component. Compose from `Surface` → `Column` → `PageHeader` → `Panel` and pull every colour from `useDS()`.

Add a page

1. `src/app/<route>/page.tsx` — Server Component; resolve the user, redirect if needed, render one client component.
2. `src/components/<feature>/<Screen>.tsx` — `"use client"`, wrap in `Surface`.
3. Add it to `BASE_LINKS` in `NavBar.tsx` if it's public.
4. Add it to `authRoutes` + `config.matcher` in `middleware.ts` if it needs a sign-in.

Add an API endpoint

1. `src/app/api/<path>/route.ts`, export `GET / POST /...` and `export const dynamic = "force-dynamic"`.
2. First lines: `const user = await getCurrentUser(req)` → 401 if absent → role check with the right predicate from `types/auth.ts` → 403.
3. Re-run the shared validators from `lib/validation.ts`.
4. Return DTOs from the feature's serializer, never raw Prisma models.

Add a database field

1. `prisma/schema.prisma` (with a doc comment — the schema is documentation here).
2. `npx prisma migrate dev --name <thing>`, commit the SQL.
3. Update the DTO in `src/types/` and the serializer in `src/lib/`.
4. Update the admin editor so it can actually be set.

Add a user role → Prisma `UserRole` enum → `USER_ROLES`, `ROLE_LABELS` and the relevant tier array in `src/types/auth.ts` → migrate.

Add an admin console section → page under `src/app/admin/` → component in `src/components/admin/` → entry in `NAV` in `AdminShell.tsx` → icon in `icons.tsx`.

Add a storage bucket → create it in Supabase → add the name to the allowlist in `src/app/api/upload/route.ts`.

10. Known rough edges

Documented so you don't rediscover them the hard way.

- **Two theming systems coexist.** `globals.css` CSS variables (legacy, consumed via Tailwind classes) and `design-system.ts` tokens (current, consumed via `useDS()`). They define overlapping colours that are *not* kept in sync. Port screens to the DS rather than editing the legacy variables.
- `proxy.ts` and `server.js` are dead code in the current setup. `proxy.ts` still gates on `role !== "ADMIN"`, which contradicts the live role model — don't copy from it.
- `prisma.ts` logs every query (`log: ["query"]`), including in production.
- **The QR scanner subsystem was removed** (commit `0ac871d`). `src/lib/crypto.ts` and the `QR_SECRET_KEY` variable are its remnants; `@zxing/browser` and `html5-qrcode` are still in `package.json`.
- **The officers roster is hardcoded** in `officers-data.ts`, unlike every other content area.
- `README.md` is still the create-next-app default. This file supersedes it.
- `src/services/` is a partially-migrated older layer. New shared logic goes in `src/lib/`.